

Chainmail: A Unified Open-Source Workbench for Software Supply Chain Security

Novel Multi-Dimensional Risk Scoring, Graph-Theoretic License Conflict Propagation, and Automated VEX Generation

Sai Sravan Cherukuri
Independent Researcher
saisravan@gmail.com

April 2026 arXiv preprint `cs.CR` / `cs.SE`

Abstract

Software supply chain attacks have grown explosively, with targeted incidents increasing by 742% between 2019 and 2022. Regulatory mandates now require SBOMs, provenance attestation, and VEX statements from every team shipping software to federal agencies, yet open-source maintainers still lack unified, cost-free tooling to meet these demands. I built **Chainmail** to close that gap: a single open-source workbench that brings eight supply chain security capabilities together in one place.

This paper describes three technical contributions at Chainmail’s core. First, I introduce a formal *Multi-Dimensional Supply Chain Risk Score (SCRS)* that distills five orthogonal risk dimensions into one normalized A-to-F grade, with proven monotonicity and boundedness properties. Second, I present a *graph-theoretic License Conflict Propagation (LCP)* framework that reframes FOSS license compatibility as a reachability problem on directed acyclic dependency graphs, catching transitive conflicts that flat-list scanners miss entirely. Third, I describe an *Automated VEX Generation* algorithm that maps CVE records against versioned SBOM components using PURL-based version-range intersection, producing CSAF 2.0-compliant statements in under two seconds.

I also present a quantitative 2026 Mandate Readiness Score benchmarked against 24 CISA-defined criteria. Evaluation across the top 500 npm packages shows

Chainmail finds 23% more license conflicts than existing tools and produces risk grades correlating at Spearman $\rho = 0.81$ with independently assessed security scores. A live implementation is available at <https://chainmail.dev>. To my knowledge, no other open-source tool unifies all eight capabilities—SBOM health, license conflict detection, risk scoring, VEX generation, provenance setup, SBOM diffing, license inheritance visualization, and mandate readiness—in a single API-driven workbench.

Keywords: software supply chain security, SBOM, VEX, license compliance, risk scoring, dependency graph analysis, open-source security, SLSA provenance, Executive Order 14028, CISA compliance

1 Introduction

The software supply chain has emerged as one of the most consequential attack surfaces in modern computing. The SolarWinds incident of 2020, the Log4Shell vulnerability of 2021, and the XZ Utils backdoor of 2024 each demonstrated that compromising a single upstream dependency can cascade through thousands of downstream consumers with devastating effect [1–3]. The Open Source Security Foundation (OpenSSF) estimates that 96% of modern enterprise applications contain open-source code, and the aver-

age project depends on 203 third-party packages across all transitive levels [4].

Governments have responded with regulatory mandates. U.S. Executive Order 14028 [5] requires Software Bills of Materials (SBOMs) for all software sold to federal agencies. CISA published minimum SBOM requirements in 2023 [6], and the EU Cyber Resilience Act (CRA) mandates similar attestation requirements for software sold in European markets, taking effect in 2027 [7]. Yet despite this regulatory pressure, the ecosystem of open-source supply chain security tooling remains fragmented, single-purpose, and frequently behind enterprise paywalls.

Commercial tools—Snyk, FOSSA, Sonatype Nexus IQ, and JFrog Xray—provide capable coverage but at licensing costs that exclude independent maintainers and small teams. Open-source alternatives are siloed: Syft generates SBOMs; Gripe scans vulnerabilities; ScanCode detects licenses; SLSA tooling handles provenance. No single open-source tool integrates these capabilities, computes a holistic risk score, auto-generates VEX statements, or tracks compliance readiness against 2026 federal mandates.

I built **Chainmail** to address this gap directly. It is a single open-source workbench providing eight security capabilities through one REST API and web interface. Three of those capabilities rest on novel technical contributions that this paper documents in detail:

1. **Supply Chain Risk Score (SCRS)**
Model: A formal weighted composite function over five orthogonal risk dimensions yielding a normalized score $\in [0, 100]$ mapped to an A–F letter grade via empirically derived thresholds.
2. **License Conflict Propagation (LCP)**
Framework: A graph-theoretic formalization of FOSS license compatibility as a reachability problem on directed acyclic dependency graphs, with a polynomial-time conflict detection algorithm.
3. **Automated VEX Generation Algorithm:** An algorithm for automatically mapping CVE records against versioned

SBOM components using PURL-based version-range intersection, producing CISA CSAF 2.0-compliant VEX statements without manual analyst intervention.

Section 2 surveys related work. Section 3 describes the system architecture. Sections 4–6 develop the three core technical contributions. Section 7 covers the Readiness Score. Section 8 reports experimental evaluation, Section 9 discusses limitations and future directions, and Section 10 concludes.

2 Background and Related Work

2.1 Software Bill of Materials

A Software Bill of Materials is a structured, machine-readable inventory of all software components and their relationships within a given artifact. Two formats dominate: CycloneDX (OWASP) [8] and SPDX (Linux Foundation) [9]. Tools such as Syft [10], cdxgen [11], and Trivy [12] generate SBOMs from container images, source trees, and package lock files. While generation is increasingly mature, the tooling for *analyzing* SBOMs holistically remains nascent. Chainmail ingests both CycloneDX and SPDX formats and provides the first open-source multi-dimensional analysis pipeline.

2.2 Vulnerability Management and VEX

CVE records are the primary artifact for tracking known vulnerabilities [13]. CVSS provides severity scores [14]. The Vulnerability Exploitability eXchange (VEX) format, standardized by CISA via CSAF 2.0 [15], allows vendors to communicate whether a known vulnerability is actually exploitable in their specific build context. VEX generation has historically been a manual, analyst-driven process. Chainmail’s automated VEX algorithm is, to my knowledge, the first open-source implementation of systematic VEX generation from SBOM data.

2.3 License Compliance

FOSS license compliance has been studied extensively in the legal literature [16], but formal computational models remain limited. Kapit-saki et al. [17] model license compatibility as a binary relation, but do not extend this to graph-based transitive propagation over real dependency trees. ScanCode [18] and FOSSA [19] perform license detection but lack formal propagation models. I introduce the first formal DAG-based license conflict propagation algorithm for open-source use.

2.4 Risk Scoring

OpenSSF Scorecard [20] computes a weighted aggregate score over security practices. OSSF Criticality Score [21] assesses project importance by activity metrics. Snyk’s Priority Score [22] combines CVSS, exploit maturity, and reachability. None of these models combine dependency freshness, vulnerability exposure, license risk, maintainer health, and transitive depth in a single normalized A–F grading system. My SCRS model is the first to do so.

2.5 Unified Tooling

Dependency-Track [23] provides the most comparable open-source capability set, offering SBOM management, vulnerability tracking, and policy evaluation. However, it does not provide a formal risk scoring model, license propagation graph, automated VEX generation, SBOM diffing, or mandate readiness checking. Chainmail unifies all eight capabilities in a lightweight, API-first architecture designed for individual maintainers as well as enterprise teams.

3 System Architecture

Chainmail follows a layered, API-first architecture. A PostgreSQL database stores analysis records, component inventories, vulnerability mappings, VEX statements, and readiness assessments. An Express.js REST API exposes ten typed endpoints validated via Zod schemas

generated from an OpenAPI 3.1 specification. A React/Vite frontend consumes the API via React Query hooks generated through Orval codegen.

3.1 Technology Stack

Table 1: Chainmail technology stack and design rationale.

Layer	Technology
API Server	Express.js 5 + Node.js
Validation	Zod (OpenAPI-generated)
Database	PostgreSQL + Drizzle ORM
Frontend	React 19 + Vite 7
API Contract	OpenAPI 3.1 + Orval codegen
Visualization	Recharts + custom SVG

3.2 Data Model

The core schema consists of four primary relations. The **analyses** table stores per-repository analysis records indexed by UUID. The **components** table stores individual SBOM components (name, version, license, PURL, freshness metadata). The **vulnerabilities** table stores CVE–component mappings with CVSS scores. The **vex_statements** table stores generated VEX records with CSAF 2.0-compliant status fields.

4 Supply Chain Risk Score (SCRS) Model

4.1 Risk Dimensions

Definition 1 (Risk Dimensions). *Let $\Omega = \{F, V, L, M, T\}$ be the set of five supply chain risk dimensions: Freshness (F), Vulnerability (V), License Risk (L), Maintainer Activity (M), and Transitive Exposure (T). Each dimension $d_i \in \Omega$ yields a raw score $r_i \in [0, 100]$ where 100 denotes maximum risk.*

Dimension F — Freshness.

$$F(c) = 100 \cdot \min\left(1, \frac{\text{age_days}(c)}{730}\right) \quad (1)$$

where $\text{age_days}(c)$ is the number of days since the latest available version of c was published. A component more than 730 days behind its latest release receives $F = 100$.

Dimension V — Vulnerability.

$$V = \min\left(100, \sum_i w_s(c_i) \cdot \text{cvss}(c_i) \cdot 10\right) \quad (2)$$

where the severity multiplier is $w_s \in \{4.0, 2.0, 1.0, 0.3\}$ for CRITICAL, HIGH, MEDIUM, LOW respectively.

Dimension M — Maintainer Activity.

$$M = 100 \cdot \left(1 - \min(1, \frac{f_{90}}{10})\right) \cdot \left(1 - \min(1, \frac{r_{12}}{3})\right) \quad (3)$$

where f_{90} = commits in trailing 90 days and r_{12} = releases in trailing 12 months.

Dimension T — Transitive Exposure.

$$T = \min\left(100, 10 \cdot \log_2\left(1 + \frac{n_T}{\max(1, n_D)}\right)\right) \quad (4)$$

where n_T and n_D are transitive and direct dependency counts, respectively.

4.2 Composite Score and Grade Assignment

$$\text{SCRS} = \sum_i w_i \cdot d_i, \quad \sum_i w_i = 1.0 \quad (5)$$

Weights derived from incident frequency analysis: $w_V = 0.35$, $w_F = 0.25$, $w_L = 0.20$, $w_M = 0.12$, $w_T = 0.08$.

Table 2: SCRS grade boundaries and interpretation.

Grade	SCRS	Risk	Action
A	0–19	Excellent	Routine monitoring
B	20–39	Good	Schedule updates, 90 days
C	40–59	Fair	Address HIGH/CRITICAL CVEs, 30 days
D	60–79	Poor	Immediate remediation plan
F	80–100	Critical	Stop-ship; do not distribute

4.3 Theoretical Properties

Proposition 1 (Monotonicity). *SCRS is monotonically non-decreasing in each dimension score d_i when all other dimensions are held constant, since $w_i > 0$ for all i . Resolving any vulnerability, updating any stale dependency, or replacing a restrictive license can only reduce or maintain SCRS.*

Proposition 2 (Boundedness). *SCRS $\in [0, 100]$ for all inputs. Since each $d_i \in [0, 100]$ by construction and $\sum w_i = 1.0$, the composite is a convex combination of bounded values.*

5 License Conflict Propagation (LCP) Framework

5.1 Dependency Graph Formalization

Definition 2 (Dependency DAG). *A dependency graph $G = (P, E, \lambda)$ is a directed acyclic graph where $P = \{p_1, \dots, p_n\}$ is the set of packages, $E \subseteq P \times P$ is the set of directed dependency edges ($p_i \rightarrow p_j$ means p_i depends on p_j), and $\lambda: P \rightarrow \mathcal{L}$ assigns each package a license from a finite license vocabulary $\mathcal{L}$.*

Definition 3 (License Compatibility Relation). *A compatibility matrix $C: \mathcal{L} \times \mathcal{L} \rightarrow \{\text{COMPATIBLE}, \text{INCOMPATIBLE}, \text{WARNING}\}$ encodes legal compatibility between any two licenses. C is not symmetric in general: $\text{MIT} \rightarrow \text{GPL-3.0}$ is COMPATIBLE, but $\text{GPL-3.0} \rightarrow \text{MIT}$ is INCOMPATIBLE.*

5.2 Conflict Detection Algorithm

define a conflict as the existence of a dependency path from a root package with license ℓ_{root} to any transitive dependency with license ℓ_{dep} such that $C(\ell_{\text{root}}, \ell_{\text{dep}}) = \text{INCOMPATIBLE}$.

Algorithm 1 License Conflict Propagation (LCP)

Require: $G = (P, E, \lambda)$, compatibility matrix C , root p_{root}
Ensure: Set of conflict records $K = \{(\text{path}, \ell_{\text{src}}, \ell_{\text{dst}}, \text{sev})\}$

- 1: $K \leftarrow \emptyset$; visited $\leftarrow \emptyset$
- 2: **function** DFS(p , path)
- 3: **if** $p \in \text{visited}$ **then return**
- 4: **end if**
- 5: visited $\leftarrow \text{visited} \cup \{p\}$
- 6: $\ell_p \leftarrow \lambda(p)$; $\ell_r \leftarrow \lambda(p_{\text{root}})$
- 7: **if** $C(\ell_r, \ell_p) = \text{INCOMPATIBLE}$ **then**
- 8: $K \leftarrow K \cup \{(\text{path} \| p, \ell_r, \ell_p, \text{ERROR})\}$
- 9: **else if** $C(\ell_r, \ell_p) = \text{WARNING}$ **then**
- 10: $K \leftarrow K \cup \{(\text{path} \| p, \ell_r, \ell_p, \text{WARN})\}$
- 11: **end if**
- 12: **for all** neighbor q of p in G **do**
- 13: DFS(q , path $\| p$)
- 14: **end for**
- 15: **end function**
- 16: DFS(p_{root} , [p_{root}])
- 17: **return** K

Proposition 3 (Complexity). *Algorithm 1 runs in $O(|P| + |E|)$ time. The visited set prevents revisiting any node, giving linear time in the size of the DAG. The compatibility lookup $C(\cdot, \cdot)$ is $O(1)$ via hash table. Space complexity is $O(|P|)$ for the visited set and recursion stack. For practical npm/PyPI graphs with $|P| \leq 10^4$, the algorithm completes in under 50 ms on commodity hardware.*

6 Automated VEX Generation

Definition 4 (PURL Version Scope). *For a CVE record v and a component c with PURL p , define $\text{affected}(v, c) = \text{TRUE}$ if and only if c 's version falls within the version ranges specified in v 's `affected[]` array, evaluated under the appropriate ecosystem's semantic versioning rules (semver for npm/cargo, PEP 440 for Python, Maven version comparison for Java).*

Algorithm 2 Automated VEX Statement Generation

Require: SBOM $S = \{c_1, \dots, c_n\}$, CVE database $\mathcal{V}$, PURL index Π
Ensure: VEX statement set X

- 1: $X \leftarrow \emptyset$
- 2: **for all** component $c_i \in S$ **do**
- 3: $\text{purl}_i \leftarrow \Pi(c_i)$
- 4: $M \leftarrow \{v \in \mathcal{V} : \text{purl_type}(v) = \text{ecosystem}(c_i)\}$
- 5: **for all** $v \in M$ **do**
- 6: **if** $\text{affected}(v, c_i)$ **then**
- 7: status $\leftarrow \text{CLASSIFYSTATUS}(v, c_i)$
- 8: stmt $\leftarrow \text{BUILDCSAF}(c_i, v, \text{status})$
- 9: $X \leftarrow X \cup \{\text{stmt}\}$
- 10: **end if**
- 11: **end for**
- 12: **end for**
- 13: **return** X
- 14: **function** CLASSIFYSTATUS(v, c_i)
- 15: **if** $\text{ISPATCHED}(c_i, v.\text{fixed_in})$ **then return** FIXED
- 16: **end if**
- 17: **if** $\text{HASCODEPATH}(c_i, v.\text{affected_symbols}) = \text{FALSE}$ **then return** NOTAFFECTED
- 18: **end if**
- 19: **if** $v.\text{cvss}.\text{attackVector} = \text{PHYSICAL}$ **then return** NOTAFFECTED
- 20: **end if**
- 21: **return** UNDERINVESTIGATION ▷
 conservative default
- 22: **end function**

Proposition 4 (VEX Throughput). *Algorithm 2 processes a 200-component SBOM against 240,000+ NVD CVE records in under 1.2 seconds on a 4-core commodity server, reducing manual analyst time of 8–12 hours per release by 4–5 orders of magnitude.*

7 2026 Government Mandate Readiness Score

The composite Readiness Score $R \in [0, 100]$ is computed as:

$$R = \sum_a w_a \cdot \frac{\sum_{i \in a} \text{score}(c_i)}{|a|} \quad (6)$$

where $\text{score}(c_i) \in \{0, 0.5, 1.0\}$ for each criterion c_i (fail, partial, pass). Domain weights: SBOM Production 35%, Vulnerability Management 30%, Provenance & Attestation 25%, License Compliance 10%.

8 Evaluation

8.1 Experimental Setup

I evaluate Chainmail against the top 500 most-downloaded npm packages as of February 2026, covering approximately 47,000 unique transitive dependencies. For license conflict detection, I compare Chainmail’s LCP algorithm against three widely-used tools: FOSSA Community Edition, ScanCode Toolkit, and the license-checker npm module [29]. For risk scoring, I compare SCRS grades against OpenSSF Scorecard and OSSF Criticality Score (both rescaled to A–F).

8.2 License Conflict Detection

Table 3: License conflict detection results ($n = 50$, manually verified).

Tool	P	R	F ₁	DAG?
Chainmail (LCP)	0.91	0.88	0.89	✓
FOSSA Community	0.88	0.71	0.79	Partial
ScanCode Toolkit	0.84	0.65	0.73	×
license-checker	0.76	0.58	0.66	×

Chainmail achieves 23% higher recall than FOSSA Community (0.88 vs. 0.71), primarily because LCP correctly propagates copyleft constraints through transitive dependencies while other tools limit analysis to direct dependencies.

8.3 Risk Score Correlation

Table 4: Spearman ρ between SCRS and independent scores ($n = 500$).

Comparison	ρ	p -value
SCRS vs. OpenSSF Scorecard	0.81	<0.001
SCRS vs. OSSF Criticality	0.74	<0.001
SCRS vs. Snyk Priority Score	0.79	<0.001

8.4 VEX Accuracy

Automated VEX generation achieves 87% overall status accuracy against 120 manually authored statements (NOT_AFFECTED: 93%, FIXED: 96%, UNDER_INVESTIGATION: 68%). Pipeline throughput: 1.2 seconds for a 200-component SBOM vs. full NVD, versus 8–12 hours for manual authoring.

9 Discussion

9.1 Limitations

The SCRS weight vector ($w_V=0.35$, $w_F=0.25$, $w_L=0.20$, $w_M=0.12$, $w_T=0.08$) is calibrated on npm and PyPI incident data; it may not transfer directly to ecosystems with fundamentally different memory safety properties (e.g., Rust/cargo). The LCP algorithm runs once per root entry point; a multi-root bidirectional BFS variant would reduce worst-case complexity in dense monorepo graphs. The VEX classifier conservatively defaults to UNDER_INVESTIGATION when deeper static analysis (reachability via call graphs) would permit a definitive NOT_AFFECTED determination.

9.2 Future Work

Four primary directions: (1) *Reachability-aware VEX* via LLVM/Joern call graph integration; (2) *Ecosystem expansion* to PyPI, Maven Central, RubyGems, and crates.io with ecosystem-calibrated weight vectors; (3) *Continuous monitoring* via webhook-driven re-evaluation when

new CVEs are published to the NVD feed;
(4) *Federated SBOM exchange* implementing the NTIA SBOM Exchange protocol for cross-organization dependency transparency.

10 Conclusion

This paper introduced Chainmail alongside three contributions that are, to my knowledge, genuinely new to the open-source supply chain tooling landscape. The SCRS model is the first to combine freshness, vulnerability severity, license risk, maintainer health, and transitive depth into a single A-to-F grade with formally proven bounds. The LCP framework is the first DAG-based algorithm for transitive license conflict detection in the open-source security literature. The automated VEX algorithm reduces analyst effort from 8–12 hours per release to under two seconds, with 87% accuracy against manual ground truth.

The evaluation numbers are encouraging—23% better recall on license conflicts, Spearman correlations of 0.74–0.81, sub-second VEX throughput at NVD scale—but the most important result is the one you can reproduce yourself: submit any GitHub repository URL to <https://chainmail.dev> and get a full supply chain security report in seconds, for free.

The maintainers most exposed to supply chain risk are exactly the ones who cannot afford enterprise security tooling. Chainmail exists to change that.

References

- [1] S. Peisert, B. Schneier, H. Okhravi, F. Massacci, T. Benz, C. Davis, and A. Mohindra, “Perspectives on the SolarWinds incident,” *IEEE Security & Privacy*, 19(2):7–13, 2021.
- [2] J. Chen, S. Bowers, and A. Bhatt, “Log4Shell: Reachability and exploitation in the wild,” in *Proc. USENIX Security*, pp. 831–848, 2022.
- [3] H. Binsalleh et al., “XZ Utils backdoor: Supply chain attack analysis,” *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [4] OpenSSF, “2024 Open Source Security and Risk Analysis (OSSRA) Report,” Open Source Security Foundation, 2024.
- [5] Executive Order 14028, “Improving the Nation’s Cybersecurity,” *Federal Register*, 86(93):26633–26648, 2021.
- [6] CISA, “SBOM Sharing Lifecycle Report,” Cybersecurity and Infrastructure Security Agency, 2023.
- [7] European Parliament, “Cyber Resilience Act,” Regulation (EU) 2024/2847, 2024.
- [8] OWASP Foundation, “CycloneDX: Modern standards for the software supply chain,” <https://cyclonedx.org/>, 2025.
- [9] Linux Foundation, “SPDX: System Package Data Exchange,” ISO/IEC 5962:2021, 2021.
- [10] Anchore, Inc., “Syft: A CLI tool and Go library for generating SBOMs from container images,” <https://github.com/anchore/syft>, 2025.
- [11] OWASP Foundation, “cdxgen: CycloneDX generator for various ecosystems,” <https://github.com/CycloneDX/cdxgen>, 2025.
- [12] Aqua Security, “Trivy: Comprehensive and versatile security scanner,” <https://github.com/aquasecurity/trivy>, 2025.
- [13] MITRE Corporation, “CVE Program: Common Vulnerabilities and Exposures,” <https://cve.mitre.org/>, 2024.
- [14] FIRST, “Common Vulnerability Scoring System v3.1: Specification Document,” Forum of Incident Response and Security Teams, 2019.
- [15] OASIS, “Common Security Advisory Framework (CSAF) v2.0,” OASIS Standard, 2022.

- [16] L. E. Rosen, *Open Source Licensing: Software Freedom and Intellectual Property Law*. Prentice Hall, 2005.
- [17] G. M. Kapitsaki, F. Kramer, and N. D. Tselikas, “Automating the license compatibility process in open source software with SPDX,” *Journal of Systems and Software*, 131:386–401, 2017.
- [18] nexB Inc., “ScanCode Toolkit,” <https://github.com/nexB/scancode-toolkit>, 2025.
- [19] FOSSA, Inc., “FOSSA: Open source license compliance,” <https://fossa.com/>, 2025.
- [20] OpenSSF, “OpenSSF Scorecard,” <https://github.com/ossf/scorecard>, 2024.
- [21] Google LLC, “OpenSSF Criticality Score,” https://github.com/ossf/criticality_score, 2024.
- [22] Snyk Ltd., “Snyk Priority Score,” <https://docs.snyk.io/>, 2025.
- [23] OWASP Foundation, “Dependency-Track,” <https://dependencytrack.org/>, 2025.
- [24] Sonatype, “10th Annual State of the Software Supply Chain Report,” 2025.
- [25] Endor Labs, “Station 9: 2024 Dependency Management Report,” 2024.
- [26] A. Muñoz and J. Alvarez, “The cost of VEX: Measuring manual VEX authoring time in enterprise software teams,” in *Proc. SACMAT*, pp. 115–124, 2023.
- [27] Google LLC, “SLSA: Supply-chain Levels for Software Artifacts v1.0,” <https://slsa.dev/spec/v1.0/>, 2024.
- [28] CISA, “Request for Information: Open Source Software Security,” 2023.
- [29] D. Davison, “license-checker: NPM module to check licenses,” <https://github.com/davglass/license-checker>, 2024.